OBJEKTNO-ORIJENTISANO PROGRAMIRANJE · SMJEROVI B, C, D

Izvođenje klasa i polimorfizam

nasljeđivanje · protected · redoslijed konstruktora · odsijecanje objekta · virtuelne metode · apstraktne klase · virtuelni destruktor

UVOD

Pregled tema

1 · 2

Nasljedivanje i pristup

Izvedena klasa, ključna riječ **protected**, šta objekat izvedene klase zapravo sadrži.

3

Konstruktori i destruktori

Ko se poziva prvi, ko posljednji, i šta se dešava kad osnovna klasa nema podrazumijevani konstruktor.

4 · 5

Imena i način izvođenja

Sakrivanje imena iz osnovne klase, tabela prava pristupa, javno naspram privatnog izvođenja.

6

Konverzija i odsijecanje

Zašto **Racun*** smije pokazivati na **StedniRacun**, i šta se gubi kad se objekat prenese po vrijednosti.

7 · 8

Polimorfizam

Statičko vezivanje kao problem, **virtual** i **override** kao rješenje, apstraktne klase.

9 · 10

Destruktor i tip u izvršavanju

Virtuelni destruktor i curenje koje nastaje bez njega, **typeid** i **dynamic_cast**.

Motivacija za izvođenje

Već imamo klasu Racun.

```
class Racun {
    std::string broj;
    double stanje;
public:
    void uplata(double iznos);
    void prikazi() const;
};
```

Ali sada imamo dodatni zahtjev: Banka traži dvije vrste računa: štedni i žiro. Pišemo ih kao dvije nezavisne klase ZiroRacun i StedniRacun, onako kako smo do sada pisali svaku klasu. Uočimo da smo dobili **dvije klase koje imaju četiri iste linije koje su pritom iste kao u klasi Racun**

```
class ZiroRacun {
    std::string broj;            // isto
    double stanje;               // isto
    double minus;       // specificno za ZiroRacun
public:
    void uplata(double iznos);   // isto
    void prikazi() const;        // isto
};
```

```
class StedniRacun {
    std::string broj;            // isto
    double stanje;               // isto
    double stopa;      // specificno za StedniRacun
public:
    void uplata(double iznos);   // isto
    void prikazi() const;        // isto
    void pripisiKamatu(); // specificno za StedniRacun
};
```

Odakle dolazi nasljeđivanje

Ponovljeni kod je znak da u sistemu imamo potrebu za nasljeđivanjem

U stvarnom sistemu

U skupu entiteta često uočimo podskup koji ima sve osobine ostalih i uz njih još neke svoje. Npr. svi studenti imaju broj indeksa i datum upisa; studenti master studija uz to imaju i naslov rada, a studenti doktorskih studija broj objavljenih radova.

U programskom kodu

Ta pojava se modeluje **nasljeđivanjem**. Važno: nasljeđivanje je relacija između **klasa**, a ne između objekata. Objekat klase **StedniRacun** jeste jedna vrsta objekta klase **Racun**.

Šta posjeduje potomak klasa

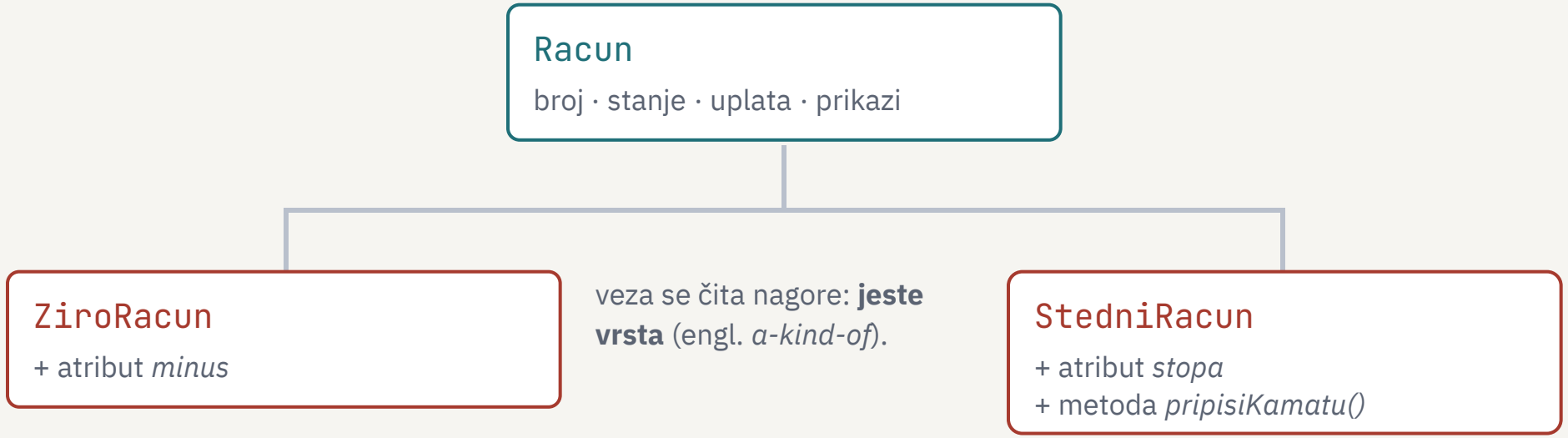
Sve osobine osnovne klase — i atribute i metode — plus one koje sama doda. Potomak nikada nema manje članova nego roditelj. Potomak klasa je specifičnija, ne manja.

Šta dobijamo nasljeđivanjem

Zajednički dio (osnovna klasa) se piše **jednom**. Pri kreiranju nove izvedene klase, osnovna klasa se ne dira: ne mijenja se, ne prevodi se ponovo i ne zna da je iko nasljeđuje. Nova vrsta računa StedniRacun je nova klasa, a ne izmjena postojećeg koda. Dakle je može je dodati bilo ko, ne mora je pisati autor klase Racun.

Dvije vrste računa: žiro račun i štedni račun

Žiro račun **jeste** račun. Štedni račun **jeste** račun. Ovakve rečenice sa „jeste“ su znak da nam treba izvođenje i da ono ima smisla.



Imena koja srećemo u literaturi

klasa od koje se polazi se naziva:	klasa koja nastaje se naziva:
osnovna klasa	izvedena klasa
nadklasa	podklasa
roditeljska klasa	klasa potomak
base class	derived class
superclass	subclass
parent	child

sve su ovo sinonimi

Na ovom kursu držimo se naziva **osnovna** i **izvedena** klasa; u literaturi na engleskom najčešće se sreću nazivi **base class** i **derived class**.
Lijeva kolona je opštija klasa, desna je specifičnija.

Izvedena klasa

osnovna klasa

```
class Racun {
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    void uplata(double iznos)
    { stanje += iznos; }
    double getStanje() const { return stanje; }
    void prikazi() const
    { std::cout << broj << ": "
                << stanje << " EUR\n"; }
};
```

ispis programa

```
501-003: 1230 EUR
```

izvedena klasa i main program

```
class StedniRacun : public Racun {
    double stopa;                 // novi atribut
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    void pripisiKamatu()          // nova metoda
    { uplata(getStanje() * stopa / 100); }
};
int main()
{
    StedniRacun s("501-003", 1000, 2.5);
    s.uplata(200);        // iz klase Racun
    s.pripisiKamatu();    // iz StedniRacun
    s.prikazi();          // iz klase Racun
}
```

Objekat **s** ima atribute **broj** i **stanje** i metode **uplata**, **getStanje** i **prikazi**, iako ih klasa **StedniRacun** nigdje ne pominje — sve to nasljeđuje iz klase **Racun**. Dodatno, objekat **s** ima atribut **stopa** i metodu **pripisiKamatu** koji su specifični za klasu StedniRacun.

Zaglavlje klase koja nasljeđuje neku klasu

```
class StedniRacun : public Racun { /* deklaracija atributa i metoda */ };
```

class StedniRacun

Ime klase koja nastaje — **izvedena klasa**. Ona je ta koju pišemo; osnovna klasa Racun već postoji.

public

Način izvođenja. Određuje kako se prava pristupa naslijeđenih članova iz osnovne klase prenose u izvedenu.

Racun

Ime **osnovne klase**. Mora biti već deklarisana. Ako ih je više, razdvajaju se zarezom — to je višestruko izvođenje.

izvođenje može ići u dubinu

```
class PremiumStedni : public StedniRacun
{ /* ... */ };
```

izvođenje može ići u širinu

```
class ZiroRacun : public Racun
{ /* ... */ };
```

Izvedena klasa može biti osnovna za **sljedeće izvođenje**, pa hijerarhija ide u više nivoa. Npr. izvedena klasa *StedniRacun* je osnovna klasa za novu klasu *PremiumStedniRacun*.

Iz jedne osnovne klase može se izvesti proizvoljan broj klasa, i one jedna o drugoj ne znaju ništa. Npr. klasa *ZiroRacun* takođe nasljeđuje klasu *Racun* kao i klasa *StedniRacun*.

Ako su atributi naslijeđeni, ne znači da su dostupni metodama izvedene klase

Trenutno metoda ***pripisiKamatu*** pristupa atributu ***stanje*** preko **javne** metode ***getStanje*** i mijenja ga preko javne metode ***uplata*** iako klasa ***StedniRacun*** posjeduje atribut ***stanje***, naslijedila ga je. Umjesto toga, kraće i prirodnije bi bilo da metoda ***pripisiKamatu*** promijeni ***stanje*** direktno. Taj kod bi izgledao ovako:

```
class Racun {
    std::string broj;
    double stanje;            // privatno
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    void uplata(double iznos)
    { stanje += iznos; }
    double getStanje() const { return stanje; }
};
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    void pripisiKamatu()
    { stanje += stanje * stopa / 100; }  // greška direktni pristup stanju
};
```

Međutim, postoji problem:

Atribut **stanje** jeste u objektu klase StedniRacun — svaki **StedniRacun** ga nosi. Ali je on u osnovnoj klasi Racun definisan kao **privatan**. Zbog toga mu metode izvedene klase ne mogu prići direktno, iako one nasljeđuju klasu Racun. Zato će ovaj kod javiti sljedeću grešku:

```
error: 'double Racun::stanje' is
       private within this context
note: declared private here
```

Zato je uveden treći nivo vidljivosti osim public i private, a to je protected

osnovna klasa

```
class Racun {
protected:                    // umjesto private
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    void uplata(double iznos)
    { stanje += iznos; }
    double getStanje() const { return stanje; }
    void prikazi() const
    { std::cout << broj << ": "
                << stanje << " EUR\n"; }
};
```

ispis programa

```
501-003: 1200 EUR
501-003: 1230 EUR
```

izvedena klasa i main program

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}

    // sada je direktan pristup stanju dozvoljen
    void pripisiKamatu()
    { stanje += stanje * stopa / 100; }};
int main()
{
    StedniRacun s("501-003", 1000, 2.5);
    s.uplata(200);
    s.prikazi();
    s.pripisiKamatu();
    s.prikazi();
}
```

Protected je sredina izmedju private i public: protected članovi klase su **dostupni u samoj klasi i u metodama izvedenih klasa, a nedostupni spolja preko objekta**.

Problem sa protected atributima: izvedena klasa može narušiti invarijantu osnovne klase

osnovna klasa čuva pravilo (invarijantu): stanje nikad ne smije biti ispod nule

```
class Racun {
protected:
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
    : broj(broj), stanje(pocetno < 0 ? 0 : pocetno) {}
    void isplata(double iznos)
    {
        if (iznos > stanje)   // čuva se invarijanta
            return;
        stanje -= iznos;
    }
    void prikazi() const
    { std::cout << broj << ": "
                << stanje << " EUR\n"; }
};
```

izvedena klasa to pravilo može da zaobiđe, tj. ne garantuje da će stanje ostati nenegativno

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    void naplatiTrosak(double iznos)
    { stanje -= iznos; }     // bez provjere
};

int main()
{
    StedniRacun s("501-003", 100, 2.5);
    s.isplata(500);      // odbijeno (isplata veća od stanja)
    s.prikazi();
    s.naplatiTrosak(500);  // prolazi
    s.prikazi();
}
```

Prvi poziv *s.isplata(500);* traži isplatu veću od stanja i bude odbijen. Drugi poziv *s.naplatiTrosak(500);* uzima isti iznos, ali kroz metodu izvedene klase. Program ispisuje -400.

Kako očuvati invarijantu i u izvedenoj klasi

prije ispravke

```
501-003: 100 EUR
501-003: -400 EUR
```

poslije ispravke

```
501-003: 100 EUR
501-003: 100 EUR
```

Rješenje

Atributi ostaju **private**, a izvedenoj klasi se otvara **protected metoda** koja radi uz provjeru: osnovna klasa zadržava kontrolu nad svojim podacima, a izvedena klasa dobija mogućnost da umanji stanje ali kontrolisano.

ispravka u osnovnoj klasi

```
class Racun {
    std::string broj;
    double stanje;        // ponovo privatno
protected:
    void promijeni(double delta)   // za potomke
    {
        if (stanje + delta < 0) return;
        stanje += delta;
    }
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    void isplata(double iznos)
    { promijeni(-iznos); }
    void prikazi() const
    { std::cout << broj << ": "
                << stanje << " EUR\n"; }
};
```

izvedena klasa sada mora umanjivati stanje kroz **protected** metodu osnovne klase (uz provjeru)

```
void naplatiTrosak(double iznos)   // u StedniRacun
{ promijeni(-iznos); }
```

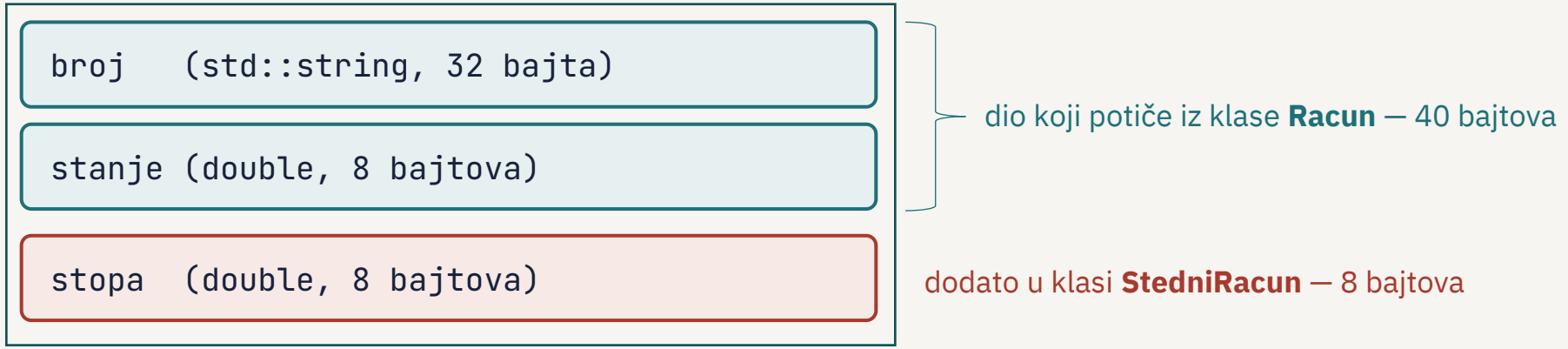
Napomena: u ostatku slajdova atributi u klasi *Racun* ostaće protected zato što primjeri treba da budu kratki, ali u realnom sistemu drđali bismo ih privatnim

2. PRISTUP

Šta objekat izvedene klase sadrži u memoriji

Naslijeđeni atributi **zauzimaju memoriju** i kad im izvedena klasa ne može prići, tj. iako su **privatni**.

objekat tipa **StedniRacun**, ukupno 48 bajtova:



Naslijeđeni dio stoji **prvi u memoriji**. Zato pokazivač na početak objekta izvedene klase ujedno pokazuje i na ispravan objekat osnovne klase.

2. PRISTUP

Ne nasljeđuju se:

Konstruktori

Ne nasljeduju se, ali se **pozivaju**. Izvedena klasa piše svoje konstruktore i kroz njih bira koji konstruktor osnovne klase će se pozvati.

Destruktor

Ne nasljeduje se, svaka klasa ima svoj destruktor, a pozivaju se redom destruktor izvedene pa destruktor osnovne klase. To prevodilac čini sam.

Operator dodjele

Izvedena klasa dobija svoj operator dodjele, koji prevodilac generiše. On poziva operator dodjele osnovne klase za naslijeđeni dio, pa kopira dodatne atribute izvedene klase.

Prijateljstvo

Nije nasljedno. Ako je neka funkcija prijatelj klase **Racun**, nije nuđno prijatelj i klase to joj ne daje pristup privatnim atributima klase **StedniRacun**.

Konstruktor osnovne klase treba pozvati u inicijalizacionoj listi izvedene klase

Lista inicijalizatora dobija još jednu stavku: **ime osnovne klase sa argumentima**, tj poziv konstruktora osnovne klase.

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje, double stopa)
        : Racun(broj, stanje),   // osnovna klasa
          stopa(stopa)           // pa svoj atribut
    {}
};
```

Inicijalizator **osnovne** klase piše se **prvi** u listi i izgleda kao poziv konstruktora: ime klase, pa argumenti. Prevodilac po tim argumentima bira koji će se konstruktor osnovne klase izvršiti.

Zašto ne setujemo naslijeđene atributa u tijelu konstruktora izvedene

U tijelu konstruktora atributi već postoje i već su inicijalizovani. Dakle dodjela vrijednosti u tijelu bi bio upis u već postojeće atribute, dakle dva upisa bismo imali.

Naslijeđeni atribut može biti deklarisan kao privatni u osnovnoj klasi, pa mu konstruktor izvedene klase ne može ni pristupiti osim preko konstruktora osnovne klase.

Redosljed poziva

Primjer: oba konstruktora i oba destruktora štampaju tekst sa cout

osnovna klasa

```
class Racun {
protected:
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje)
    { std::cout << "Racun: konstruktor\n"; }
    ~Racun()
    { std::cout << "Racun: destruktor\n"; }
};
```

izvedena klasa

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa)
    { std::cout << "StedniRacun: konstruktor\n"; }
    ~StedniRacun()
    { std::cout << "StedniRacun: destruktor\n"; }
};
```

```
int main()
{
    std::cout << "ulazimo u blok\n";
    {
        StedniRacun s("501-003", 1000, 2.5);
    }
    std::cout << "izasli smo iz bloka\n";
}
```

ispis programa

```
ulazimo u blok
Racun: konstruktor
StedniRacun: konstruktor
StedniRacun: destruktor
Racun: destruktor
izasli smo iz bloka
```

Redosljed poziva

Redoslijed poziva konstruktora kada se pravi objekat izvedene klase

Prvo konstruktor osnovne klase, zatim inicijalizacija atributa izvedene klase, pa tek onda tijelo konstruktora izvedene klase. Izvedena klasa se oslanja na osnovnu, pa osnovna mora biti gotova prva.

Redoslijed poziva destruktora kada se uništava objekat izvedene klase

Obrnut od konstruktora: destruktor izvedene klase, pa destruktor osnovne. U destruktoru izvedene klase **ne** pozivamo destruktor osnovne — prevodilac to radi sam.

Kad se inicijalizator osnovne klase izostavi

Ako osnovna klasa nema podrazumijevani konstruktor, i ako bismo izostavili poziv konstruktora osnovne klase u inicijalizaciji izvedene, i pokušali da dodijelimo vrijednosti u tijelu konstruktora izvedene klase, kao što bi se radilo u običnoj funkciji, dobili bismo grešku.

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : stopa(stopa)        // bez poziva Racun(...)
    {
        this->broj = broj;
        this->stanje = stanje;
    }
};
```

Bez inicijalizatora prevodilac pokušava da pozove **podrazumijevani** konstruktor osnovne klase, onaj bez argumenata. Klasa **Racun** ga nema jer je definisala konstruktor sa 2 argumenta, pa prevođenje staje prije nego što se dođe do tijela. Dobijamo grešku:

```
error: no matching function for call
       to 'Racun::Racun()'
note: candidate expects 2 arguments,
      0 provided
```

Zaključak: poziv konstruktora osnovne klase u inicijalizacionoj listi izvedene klase je obavezan ako osnovna klasa nema podrazumijevani konstruktor a preporuka je i za slučaj kada ga ima.

4. SAKRIVANJE IMENA

Isto ime u izvedenoj klasi

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    void prikazi() const       // isto ime kao u Racun
    {
        Racun::prikazi();      // verzija osnovne klase
        std::cout << "  stopa " << stopa << "%\n";
    }
};

int main()
{
    StedniRacun s("501-003", 1000, 2.5);
    s.prikazi();         // StedniRacun::prikazi
    s.Racun::prikazi();  // izričito metoda osnovne klase
}
```

osnovna klasa

```
class Racun {
protected:
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    void uplata(double iznos)
    { stanje += iznos; }
    double getStanje() const { return stanje; }
    void prikazi() const
    { std::cout << broj << ": "
                << stanje << " EUR\n"; }
};
```

ispis programa

```
501-003: 1000 EUR
  stopa 2.5%
501-003: 1000 EUR
```

Metoda izvedene klase sa **istim imenom sakriva** metodu osnovne klase, ne briše je.

Operatorom **::** uvijek možemo pozvati verziju iz osnovne klase

Sakriva se cijelo ime metode, ne samo konkretan potpis metode

osnovna klasa ima uplatu sa jednim argumentom

```
class Racun {
protected:
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    void uplata(double iznos) { stanje += iznos; }

};
```

izvedena dodaje uplatu sa dva argumenta

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    void uplata(double iznos, std::string opis);
};

int main()
{
    StedniRacun s("501-003", 1000, 2.5);
    s.uplata(200);   // jedan argument, GREŠKA
}
```

```
error: no matching function for call to
       'StedniRacun::uplata(int)'
note: candidate:
      'void StedniRacun::uplata(double,
       std::string)'
note:   candidate expects 2 arguments,
        1 provided
```

Zašto prevodilac ne ponudi verziju iz osnovne klase

Traženje imena počinje od klase objekta i **staje čim se ime nađe**. Ime **uplata** postoji u klasi **StedniRacun**, pa se u klasi **Racun** više i ne gleda. Zato u poruci stoji samo jedan kandidat: verzija iz osnovne klase nije nedostupna, nego **nevidljiva**. Sakrivanje radi po imenu, ne po potpisu — dovoljno je da se ime poklopi.

Kako vratiti sakriveno ime

Ključna riječ *using*

Deklaracija ***using Racun::uplata;*** uvodi ime iz osnovne klase u izvedenu. Obje verzije su sada dostupne preko objekta izvedene klase, pa prevodilac bira po **broju i tipu argumenata**, kao kod običnog preklapanja.

Naredba ***using Racun::uplata;*** uvodi u izvedenu klasu sve verzije tog imena iz osnovne klase, čak iako ih ima više.

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    using Racun::uplata;    // uvedi i verziju
                            // iz osnovne klase
    void uplata(double iznos, std::string opis)
    {
        std::cout << "uplata: " << opis << "\n";
        Racun::uplata(iznos);
    }
};
```

```
int main()
{
    StedniRacun s("501-003", 1000, 2.5);
    s.uplata(200);            // OK osnovna klasa
    s.uplata(300, "kamata");  // OK izvedena klasa
    s.prikazi();
}
```

ispis programa

```
uplata: kamata
501-003: 1500 EUR
```

Ključna riječ ispred imena osnovne klase: način nasljeđivanja

```
class StedniRacun : public Racun    { /* ... */ };
class StedniRacun : protected Racun { /* ... */ };
class StedniRacun : private Racun   { /* ... */ };

class  StedniRacun : Racun { /* ... */ };  // kod class po default-u je private izvođenje
struct StedniRacun : Racun { /* ... */ };  // kod struct po default-u je public izvođenje
```

Način izvođenja određuje kako izvedena klasa može da nudi naslijeđene članove ostatku koda.

Način izvođenja nema nikakvog uticaja na pravo pristupa naslijeđenim članovima **unutar metoda izvedene klase**.

public izvođenje

Javni članovi osnovne klase ostaju javni i u izvedenoj. *Public* način nasljeđivanja je jedini od tri načina koji realizuje odnos ***jeste vrsta***.

protected izvođenje

Javni članovi postaju zaštićeni u izvedenoj klasi. Dostupni su daljim potomcima, ali ne i korisniku izvedene klase.

private izvođenje

Sve što je naslijeđeno postaje privatno u izvedenoj klasi. Spolja, preko objekta izvedene klase se ne vidi ništa što je naslijeđeno iz osnovne klase.

5. NAČIN IZVOĐENJA

Tabela prava pristupa

Izvođenje	javni član postaje	zaštićeni postaje	privatni član
public	javni	zaštićeni	nedostupan
protected	zaštićeni	zaštićeni	nedostupan
private	privatni	privatni	nedostupan

Pravo pristupa može se samo sniziti, nikad povisiti — privatni član osnovne klase ne može postati zaštićen ni javan.

Tabela govori o pristupu **spolja**, preko objekta izvedene klase, a ne o tome kojim članovima mogu pristupati **metode izvedene klase.**

Pristup unutar izvedene klase i pristup spolja preko objekta izvedene klase

```
class Racun {
    std::string broj;          // privatni
protected:
    double stanje;             // zasticeni
public:
    void uplata(double iznos) // javni
    { stanje += iznos; }
};

class StedniRacun : protected Racun {
public:
    void test()
    {
        stanje += 1;
        uplata(10);
        broj = “501-005“;
    }
};
```

```
int main()
{
    StedniRacun s;
    s.test();
    s.uplata(10);
}
```

Dva poziva metode *uplata*

Prvi poziv je **unutar metode** izvedene klase i on prolazi.

Drugi je u **main-u, preko objekta izvedene klase** i on ne prolazi: zaštićeno izvođenje zatvorilo je metodu **uplata** prema spolja.

Pristup unutar izvedene klase i pristup spolja preko objekta izvedene klase

```
class Racun {
    std::string broj;          // privatni
protected:
    double stanje;             // zasticeni
public:
    void uplata(double iznos) // javni
    { stanje += iznos; }
};

class StedniRacun : protected Racun {
public:
    void test()
    {
        stanje += 1;    // OK, zasticeni clan
        uplata(10);     // OK, javna metoda
        broj = “501-005“; // greska, privatni clan
    }
};
```

```
int main()
{
    StedniRacun s;
    s.test();
    s.uplata(10);   // greska, pristup spolja
}
```

```
error: 'double Racun::tajna' is private
       within this context
error: 'void Racun::uplata(double)' is
       inaccessible within this context
```

Dva poziva metode *uplata*

Prvi poziv je **unutar metode** izvedene klase i on prolazi.

Drugi je u **main-u, preko objekta izvedene klase** i on ne prolazi: zaštićeno izvođenje zatvorilo je metodu **uplata** prema spolja.

Javno izvođenje znači *jeste*
Privatno izvođenje znači *ima*

```
class StedniRacun : private Racun {  // privatno izvođenje
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    void pripisiKamatu()
    {
        stanje += stanje * stopa / 100;  // OK
        prikazi();                       // OK
    }
};

int main()
{
    StedniRacun s("501-003", 1000, 2.5);
    s.pripisiKamatu();   // OK, svoja javna metoda
    s.uplata(200);       // greska
}
```

osnovna klasa

```
class Racun {
protected:
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    void uplata(double iznos)
    { stanje += iznos; }
    double getStanje() const { return stanje; }
    void prikazi() const
    { std::cout << broj << ": "
                << stanje << " EUR\n"; }
};
```

Privatno izvođenje znači da izvedena klasa **koristi** osnovnu, a ne da **jeste** njena vrsta. **Isto se postiže atributom tipa Racun** unutar klase, bez izvođenja, što je čitljivije, pa se u praksi bira to. Privatno izvođenje se koristi rijetko.

```
error: 'void Racun::uplata(double)' is
       inaccessible within this context
error: 'Racun' is not an accessible
       base of 'StedniRacun'
```

Primjer sa dvije izvedene klase

Obje izvedene klase koriste isti obrazac za metodu *prikazi*: sopstvena metoda **prikazi** poziva **Racun::prikazi** i dopunjava ispis svojim dijelom.

osnovna klasa

```
class Racun {
protected:
    std::string broj;
    double stanje;
public:
    Racun(std::string broj, double stanje)
        : broj(broj), stanje(stanje) {}
    void prikazi() const
    { std::cout << broj << ": "
                << stanje << " EUR\n"; }
};
```

```
int main()
{
    ZiroRacun   z("501-001", 1000, 500);
    StedniRacun s("501-003", 2000, 2.5);
    z.prikazi();
    s.prikazi();
}
```

ispis programa

```
501-001: 1000 EUR
  minus do 500
501-003: 2000 EUR
  stopa 2.5%
```

prva izvedena klasa

```
class StedniRacun : public Racun {
    double stopa;
public:
    StedniRacun(std::string broj, double stanje,
                double stopa)
        : Racun(broj, stanje), stopa(stopa) {}
    void prikazi() const
    { Racun::prikazi();
      std::cout << "  stopa " << stopa << "%\n"; }
};
```

druga izvedena klasa

```
class ZiroRacun : public Racun {
    double minus;
public:
    ZiroRacun(std::string broj, double stanje,
              double minus)
        : Racun(broj, stanje), minus(minus) {}
    void prikazi() const
    { Racun::prikazi();
      std::cout << "  minus do " << minus << "\n"; }
};
```

Pokazivač na osnovnu klasu

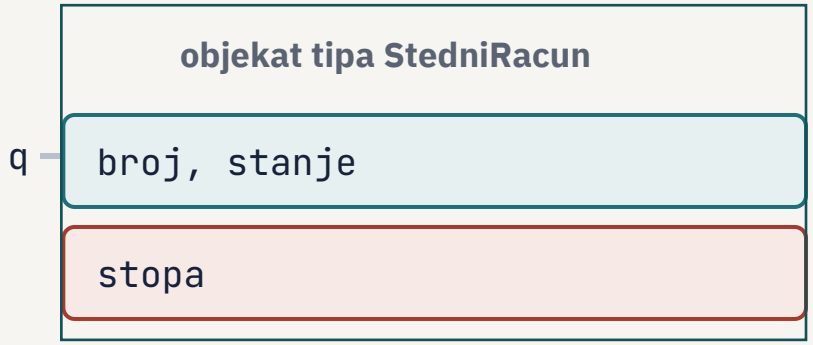
```
ZiroRacun   z("501-001", 1000, 500);
StedniRacun s("501-003", 2000, 2.5);

Racun* p = &z;       // dozvoljeno ako je nasljeđivanje public (z jeste Racun)
Racun* q = &s;       // dozvoljeno ako je nasljeđivanje public (s jeste Racun)

// obrnuto ne radi:
// StedniRacun* r = p;   // greska (p nije štedni racun)
```

Zašto ovo radi

Naslijeđeni dio je na početku objekta. Adresa objekta **StedniRacun** je ujedno i adresa ispravnog objekta **Racun** u njemu.



Odsijecanje objekta

Objektu osnovne klase smije se dodijeliti objekat izvedene. Pitanje je **šta od njega ostane**.

```
void ispisi(Racun r)        // prenos po vrijednosti
{
    std::cout << "u funkciji, velicina: "
              << sizeof(r) << "\n";
    r.prikazi(); // poziva se verzija iz klase Racun
}

int main()
{
    StedniRacun s("501-003", 1000, 2.5);
    std::cout << "u mainu, velicina:    "
              << sizeof(s) << "\n";
    s.prikazi();
    ispisi(s);
}
```

ispis programa

```
u mainu, velicina:    48
501-003: 1000 EUR
  stopa 2.5%
u funkciji, velicina: 40
501-003: 1000 EUR
```

Parametar **r** je tipa **Racun**, pa se pravi kopija od 40 bajtova. Atribut **stopa** nema gdje da stane i onda nestaje. To je **odsijecanje** (engl. *object slicing*).

Referenca ne pravi kopiju

```
void ispisi(const Racun& r)   // prenos po referenci
{
    std::cout << "adresa u funkciji: " << &r << "\n";
    r.prikazi();
}

int main()
{
    StedniRacun s("501-003", 1000, 2.5);
    std::cout << "adresa u mainu:    " << &s << "\n";
    ispisi(s);
}
```

ispis programa

```
adresa u mainu:    0x7fffc5464ed0
adresa u funkciji: 0x7fffc5464ed0
501-003: 1000 EUR
```

Iste adrese: nema kopije, objekat je i dalje **StedniRacun**, sa stopom u sebi. Pa zašto se onda i dalje ispisuje samo verzija iz klase **Racun**? To je statičko vezivanje – verzija metode se određuje po tipu pokazivača a ne po tipu objekta.